Class: BE **Subject: Applied Data Science Lab** **Sem-VIII**

Experiment No 6

Course Outcome: CO4 **Blooms Level: L3**

Aim: Implement SMOTE technique to generate synthetic data to solve the problem of class imbalance.

Abstract: Implement the Synthetic Minority Oversampling Technique (SMOTE) to address the problem of class imbalance in a classification dataset. Use the imbalanced **Student Admission dataset** containing attributes such as admit status, GRE score, GPA, and rank. Analyze the class distribution of the dataset, apply the SMOTE technique to generate synthetic samples for the minority class, and create a balanced dataset. Train a classification model using the original and the balanced datasets, compare the model performance, and evaluate the results using appropriate performance evaluation metrics to assess the impact of SMOTE on classification accuracy and class prediction performance.

Theory:

The most widely used approach to synthesizing new examples is called the Synthetic Minority Oversampling TEchnique, or SMOTE for short. SMOTE works by selecting examples that are close in the feature space, drawing a line between the examples in the feature space and drawing a new sample at a point along that line. Specifically, a random example from the minority class is first chosen. Then k of the nearest neighbors for that example are found (typically k=5). A randomly selected neighbor is chosen and a synthetic example is created at a randomly selected point between the two examples in feature space. The advantage of SMOTE is that you are not generating duplicates, but rather creating synthetic data points that are slightly different from the original data points.

The SMOTE algorithm works as follows:

- Draw a random sample from the minority class.
- For the observations in this sample, identify the k nearest neighbors.
- Take one of those neighbors and identify the vector between the current data point and the selected neighbor.
- Multiply the vector by a random number between 0 and 1.
- To obtain the synthetic data point, add this to the current data point.

This operation is very much like slightly moving the data point in the direction of its neighbor. This way, you make sure that your synthetic data point is not an exact copy of an existing data point while making sure that it is also not too different from the known observations in your minority class.

Program:

In [1]:
```
import pandas as pd
import pandas as pd
import seaborn as sns
```

```
C:\Users\Ramya\Anaconda3\lib\site-packages\statsmodels\tools\_testing.py:19: FutureWarning: pandas.ut
il.testing is deprecated. Use the functions in the public API at pandas.testing instead.
  import pandas.util.testing as tm
```

In [2]:
```
dataset=pd.read_excel(r'C:\Users\Ramya\Desktop\ads sttp data\imbalance_smote\Admission_St.xlsx')
```

In [3]:
```
dataset
```

Out[3]:

	Admit	GRE	GPA	RANK
0	0	380	3.61	3
1	1	660	3.67	3
2	1	800	4.00	1
3	1	640	3.19	4
4	0	520	2.93	4
...	...	...	...	...
395	0	620	4.00	2
396	0	560	3.04	3
397	0	460	2.63	2
398	0	700	3.65	2
399	0	600	3.89	3

400 rows × 4 columns

In [4]:
```
X=dataset.iloc[:,1:4]
```

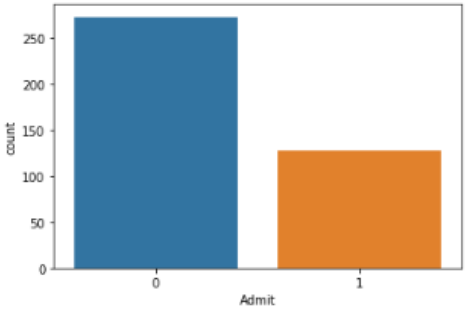
In [5]:
```
Y=dataset.iloc[:,0:1]
```

In [6]:
```
Y.value_counts()
```

Out[6]:
```
Admit
0        273
1        127
dtype: int64
```

In [7]:
```
sns.countplot(x="Admit",data=dataset)
```

Out[7]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x294b4d74808>
```



In [8]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.3,random_state=0)
```

In [9]:
```
len(X_train)
len(Y_train)
```

Out[9]:
```
280
```

In [10]:
```
len(X_test)
len(Y_test)
```

Out[10]:
```
120
```

In [11]:
```
from imblearn.over_sampling import RandomOverSampler
ros=RandomOverSampler()
X_ros,Y_ros=ros.fit_resample(X_train,Y_train)
```

In [12]:
```
len(Y_ros)
```

Out[12]:
```
382
```

In [13]:
```
Y_ros.value_counts()
```

Out[13]:
```
Admit
0        191
1        191
dtype: int64
```

In [14]:
```
from imblearn.under_sampling import RandomUnderSampler
rus=RandomUnderSampler()
X_rus,Y_rus=rus.fit_resample(X_train,Y_train)
```

In [15]:
```
len(Y_rus)
```
Out[15]:
```
178
```

In [16]:
```
Y_rus.value_counts()
```
Out[16]:
```
Admit
0        89
1        89
dtype: int64
```

In [17]:
```
from imblearn.over_sampling import SMOTE
X_smote,Y_smote=SMOTE(k_neighbors=3).fit_resample(X_train,Y_train)
Y_smote.value_counts()
```
Out[17]:
```
Admit
0        191
1        191
dtype: int64
```

Conclusion: SMOTE algorithm is used as a solution for imbalanced data in the experiment. SMOTE is an intelligent alternative to oversampling: rather than creating duplicates of the minority class, it creates synthetic data points that are relatively similar to the original ones.

Exercise 1:
Dataset: https://github.com/MedicharlaKarthik/Credit-Card-Fraud-Detection?

Abstract: The Credit Card Fraud Detection dataset contains transactions made by credit card holders, where fraudulent transactions form a very small minority compared to legitimate transactions, resulting in a highly imbalanced dataset. Perform exploratory data analysis and preprocessing to understand the class distribution. Build a classification model to detect fraudulent transactions and evaluate its performance using metrics such as confusion matrix, precision, recall, F1-score, and accuracy. Apply SMOTE (Synthetic Minority Over-sampling Technique) to balance the dataset and retrain the model. Compare model performance before and after applying SMOTE and interpret the impact of class imbalance handling on model evaluation metrics.

.

Exercise 2:
Dataset:
https://uci-ics-mlr-prod.aws.uci.edu/dataset/17/breast%2Bcancer%2Bwisconsin%2Bdiagnostic?

Abstract: The Breast Cancer Wisconsin dataset contains diagnostic measurements of breast cell nuclei used to classify tumors as benign or malignant. The dataset exhibits class imbalance that may affect classification performance. Perform data preprocessing and exploratory analysis, followed by training a classification model for tumor prediction. Evaluate model performance using confusion matrix, accuracy, precision, recall, and F1-score. Apply SMOTE to balance the minority class and compare the performance of the model before and after resampling. Analyze the improvement in classification performance and discuss the effectiveness of SMOTE in handling imbalanced datasets.